

Les exemples décrivent la même bibliothèque que `3-active-record.pdf` : des auteurs (`Author`), leurs livres (`Book`), chacun en un nombre limité d'exemplaires, des membres (`Member`) et leurs emprunts (`Loan`).

♦ Step 10 : les tests

Un test de modèle

Minitest est fourni avec Rails. Les tests sont dans `test/`, un fichier par classe testée :

```
# test/models/loan_test.rb
require "test_helper"

class LoanTest < ActiveSupport::TestCase
  test "un livre sans exemplaire disponible ne peut pas être emprunté" do
    book = books(:germinal)
    book.update!(copies: 1)
    loan = Loan.new(book: book, member: members(:denis))

    assert_not loan.valid?
    assert_includes loan.errors[:book], "n'a plus d'exemplaire disponible"
  end
end
```

`test "..."` définit un test. Chacun prépare une situation, agit, puis vérifie le résultat avec des assertions.

Les fixtures

Les fixtures sont les données de test : un fichier YAML par table, dans `test/fixtures/`, une entrée nommée par ligne.

```
# test/fixtures/authors.yml
zola:
  name: Émile Zola

# test/fixtures/books.yml : 2 exemplaires, dont un prêté à Camille (voir loans.yml)
germinal:
  author: zola           # renvoie à la fixture zola de authors.yml
  title: Germinal
  copies: 2
  status: available      # le nom de la valeur d'enum
```

```
# test/fixtures/members.yml
camille:
  name: Camille Durand
  email: camille@example.test

denis:
  name: Denis Petit
  email: denis@example.test

# test/fixtures/loans.yml
camille_germinal:
  book: germinal
  member: camille
  created_at: <%= 1.week.ago %> # l'ERB calcule une date relative
```

`books(:germinal)` renvoie le livre de l'entrée `germinal`. Les fixtures sont chargées dans une base à part, `storage/test.sqlite3`, et chaque test est annulé à la fin : les tests ne se gênent pas entre eux.

Les assertions courantes

Assertion	Réussit si
<code>assert valeur</code> , <code>assert_not valeur</code>	la valeur est <code>truthy</code> , ou <code>falsy</code>
<code>assert_equal attendu, obtenu</code>	les deux valeurs sont égales
<code>assert_nil valeur</code>	la valeur est <code>nil</code>
<code>assert_includes collection, élément</code>	la collection contient l'élément
<code>assert_difference("Loan.count") do ... end</code>	<code>Loan.count</code> a augmenté de 1 pendant le bloc ; <code>assert_difference("Loan.count", -1)</code> pour une baisse
<code>assert_no_difference("Loan.count") do ... end</code>	<code>Loan.count</code> n'a pas changé pendant le bloc
<code>assert_redirected_to book_url(book)</code>	la réponse redirige vers cette URL
<code>assert_select "td", "eve@example.test"</code>	le HTML de la réponse contient une cellule avec ce texte

Relire en base : `reload`

```
test "rendre un exemplaire repasse le livre à disponible" do
  book = books(:germinal)
  book.loans.create!(member: members(:denis)) # le dernier exemplaire part : on_loan

  loans(:camille_germinal).destroy

  assert book.reload.available?
end
```

`loans(:camille_germinal)` charge son livre dans un autre objet Ruby : le callback `after_destroy` modifie cet objet-là, pas la variable `book`. `reload` relit la ligne en base. Sans lui, le test échoue.

Un test d'intégration

Un test d'intégration envoie de vraies requêtes à l'application, sans navigateur : routeur, contrôleur, modèle et vue.

```
# test/controllers/loans_controller_test.rb
require "test_helper"

class LoansControllerTest < ActionDispatch::IntegrationTest
  test "emprunter un livre" do
    book = books(:germinal)

    assert_difference("Loan.count") do
      post book_loans_url(book),
        params: { member: { name: "Ève Leroy", email: "eve@example.test" } }
    end

    assert_redirected_to book_url(book)
    follow_redirect!
    assert_select "td", "eve@example.test"
  end

  test "rendre un livre" do
    loan = loans(:camille_germinal)

    assert_difference("Loan.count", -1) do
      delete loan_url(loan)
    end

    assert_redirected_to book_url(loan.book)
  end
end
```

`params:` reprend la forme des champs du formulaire (`member[name]`, `member[email]`). `follow_redirect!` suit la redirection, pour vérifier la page suivante. Après la requête, `flash[:alert]` donne le message d'erreur éventuel.

Lancer les tests, lire un échec

```
bin/rails test           # toute la suite
bin/rails test test/models  # un dossier
bin/rails test test/models/loan_test.rb  # un fichier
bin/rails test test/models/loan_test.rb:12  # le test de la ligne 12
```

```
Failure:
BookTest#test_compte_les_exemplaires_disponibles [test/models/book_test.rb:5]:
Expected: 2
Actual: 1

bin/rails test test/models/book_test.rb:4
```

Dans l'ordre : le test, le fichier et la ligne de l'assertion qui échoue, la valeur attendue et la valeur obtenue, puis la commande qui relance ce seul test. Un `assert` qui reçoit `false` affiche `Expected false to be truthy.`

♦ Step 11 : association polymorphe

Une table pour deux modèles

Un avis peut porter sur un livre ou sur un auteur. Plutôt qu'une table d'avis par modèle, une seule table, dont chaque ligne dit à quel

type d'objet elle appartient :

```
bin/rails generate model Review body:text "reviewable:references{polymorphic}"
```

La migration contient `t.references :reviewable, polymorphic: true, null: false` : deux colonnes, `reviewable_type` (le nom de la classe, "Book" ou "Author") et `reviewable_id` (l'id dans la table correspondante), et un index sur le couple.

```
class Review < ApplicationRecord
  belongs_to :reviewable, polymorphic: true
end

class Book < ApplicationRecord # de même dans Author
  has_many :reviews, as: :reviewable, dependent: :destroy
end
```

```
review = germinal.reviews.create!(body: "Le meilleur roman de la série")
review.reviewable_type # => "Book"
review.reviewable_id   # => 1
review.reviewable      # => le livre : Rails lit reviewable_type pour choisir la classe

germinal.reviews
# SELECT "reviews".* FROM "reviews"
# WHERE "reviews"."reviewable_id" = 1 AND "reviews"."reviewable_type" = 'Book'
```

`as: :reviewable` indique à `has_many` de chercher par les deux colonnes.

Un contrôleur pour deux parents

```
resources :authors do
  resources :reviews, only: :create # POST /authors/:author_id/reviews
end
resources :books do
  resources :reviews, only: :create # POST /books/:book_id/reviews
end
```

Deux URL, un seul contrôleur. Le paramètre présent dit quel parent charger :

```
class ReviewsController < ApplicationController
  before_action :set_reviewable

  def create
    review = @reviewable.reviews.build(review_params)

    if review.save
      redirect_to @reviewable, notice: "Avis ajouté."
    else
      redirect_to @reviewable, alert: "Un avis ne peut pas être vide."
    end
  end

  private

  def set_reviewable
    if params[:book_id]
      @reviewable = Book.find(params[:book_id])
    else
      @reviewable = Author.find(params[:author_id])
    end
  end

  def review_params
    params.expect(review: [ :body ])
  end
end
```

`create` n'a pas besoin de savoir si `@reviewable` est un livre ou un auteur : les deux ont `reviews`, et `redirect_to` trouve l'URL de chacun. Même chose dans un partial partagé, rendu avec `render "reviews/reviews", reviewable: @book : form_with model: [reviewable, Review.new]` envoie le formulaire à `/books/1/reviews` ou à `/authors/1/reviews`.

Pas de clé étrangère

`reviewable_id` pointe tantôt vers `books`, tantôt vers `authors` : aucune clé étrangère n'est possible, et la base ne vérifie pas que le livre ou l'auteur existe. L'intégrité repose sur le code : `belongs_to` le vérifie à l'enregistrement (« Reviewable doit exister »), et `dependent: :destroy` supprime les avis avec leur livre.

Un concern pour le code partagé

`Book` et `Author` contiennent la même ligne `has_many :reviews, as: :reviewable, dependent: :destroy`. Un concern la range dans un module, écrit une seule fois :

```
# app/models/concerns/reviewable.rb
module Reviewable
  extend ActiveSupport::Concern

  included do
    has_many :reviews, as: :reviewable, dependent: :destroy
  end

  def reviewed?
    reviews.any?
  end
end

class Book < ApplicationRecord # de même dans Author
  include Reviewable
end
```

`has_many` est une méthode de classe : elle doit s'exécuter dans la classe qui inclut le module. C'est le rôle de `included do`, dont le bloc s'exécute dans `Book`, puis dans `Author`, au moment du `include`. Les méthodes écrites hors du bloc deviennent des méthodes d'instance, comme dans tout module. `Book.ancestors` liste `Reviewable` juste après `Book`.

Les concerns de modèles sont dans `app/models/concerns/`, ceux de contrôleurs dans `app/controllers/concerns/`.

♦ Step 12 : Turbo Drive et Turbo Frames

Turbo Drive

Turbo est installé par défaut (`app/javascript/application.js` importe `@hotwired/turbo-rails`). Turbo Drive intercepte les clics sur les liens et les envois de formulaires, fait la requête lui-même et remplace le `<body>` par celui de la réponse : la page n'est jamais rechargée en entier.

Après l'envoi d'un formulaire, Turbo attend une redirection (le succès) ou un statut d'erreur (l'échec). D'où les deux branches d'une action `create` ou `update` :

```
if @book.save
  redirect_to @book, notice: "Livres créé."
else
  render :new, status: :unprocessable_content # 422 : Turbo affiche le formulaire
end
```

Sans le statut 422, Turbo n'affiche rien et signale dans la console du navigateur : `Form responses must redirect to another location.`

Un frame : une zone qui se met à jour seule

```
<%= turbo_frame_tag "book_details" do %>
  <h1><%= @book.title %></h1>
  <p><%= pluralize(@book.copies, "exemplaire", plural: "exemplaires") %></p>

  <%= link_to "Modifier", edit_book_path(@book) %>
<% end %>
```

```
<%=# app/views/books/edit.html.erb %>
<%= turbo_frame_tag "book_details" do %>
  <%= render "form", book: @book %>

  <%= link_to "Annuler", @book %>
<% end %>
```

`turbo_frame_tag "book_details"` produit une balise `<turbo-frame id="book_details">`. Un lien ou un formulaire placé **dans** un frame ne change pas de page : Turbo fait la requête, cherche dans la réponse le frame qui porte le même `id`, et remplace le contenu du frame affiché par le sien. Le reste de la réponse est ignoré.

Ici, « Modifier » affiche le formulaire à la place des informations ; l'enregistrement redirige vers `show`, dont Turbo ne garde que le frame ; une erreur réaffiche le formulaire sur place. Le contrôleur ne change pas.

Sortir d'un frame

Si la réponse ne contient pas le frame attendu, Turbo affiche « Content missing ». Pour qu'un lien placé dans un frame change toute la page :

```
<%= link_to "Voir l'auteur", @book.author, data: { turbo_frame: "_top" } %>
```

Un frame ne met à jour que lui-même : ce qui est affiché ailleurs dans la page (un titre, un compteur, un message du layout) garde son ancienne valeur.

♦ Step 13 : Turbo Streams

Plusieurs zones, une réponse

Un Turbo Stream est une réponse faite d'instructions. Chacune vise un élément de la page par son `id` :

```
<td id="available_copies"><%= @book.available_copies %></td>
<tbody id="loans"><%= render @book.loans %></tbody>
```

Le contrôleur répond en stream quand la requête vient de Turbo, et garde une réponse HTML pour les autres (tests, requêtes sans JavaScript) :

```
def create
  # ... @loan est construit et enregistré
  respond_to do |format|
    format.turbo_stream # rend create.turbo_stream.erb
    format.html { redirect_to @book, notice: "Emprunt enregistré." }
  end
end
```

```
<%=# app/views/loans/create.turbo_stream.erb %>
<%= turbo_stream.append "loans", @loan %>
<%= turbo_stream.update "available_copies", @book.available_copies %>
```

La réponse est du HTML, découpé en instructions :

```
<turbo-stream action="append" target="loans">
  <template><tr id="loan_12">...</tr></template>
</turbo-stream>
<turbo-stream action="update" target="available_copies">
  <template>1</template>
</turbo-stream>
```

`turbo_stream.append "loans", @loan` rend le partial `loans/_loan`, comme `render @loan`.

Action	Effet
<code>append</code> / <code>prepend</code>	ajoute à la fin / au début de la cible
<code>update</code>	remplace le contenu de la cible, garde la balise
<code>replace</code>	remplace la cible elle-même, balise comprise
<code>remove</code>	supprime la cible : <code>turbo_stream.remove @loan</code> vise l'élément <code>loan_12</code>

Pour rendre un partial dans une instruction : `turbo_stream.replace "loan_form", partial: "loans/form", locals: { book: @book }`. Une instruction dont la cible n'existe pas dans la page est ignorée.

Prévenir les autres navigateurs

Une page ouverte ailleurs n'a rien demandé : le serveur doit lui pousser le changement, par une connexion qui reste ouverte (un WebSocket, géré par Action Cable).

```
<%# app/views/books/show.html.erb : la page s'abonne aux messages de ce livre %>
<%= turbo_stream_from @book %>
```

```
class Loan < ApplicationRecord
  belongs_to :book
  broadcasts_refreshes_to :book      # après chaque création, modification ou suppression
end
```

Les pages abonnées reçoivent une instruction `refresh` et rechargent leur contenu. `broadcasts_refreshes_to` pose des callbacks `after_commit` : comme ceux du step 08, ils s'exécutent d'où que vienne l'écriture.

♦ Step 14 : Active Storage

Installer

```
bin/rails active_storage:install
bin/rails db:migrate
```

La migration crée trois tables, qui servent à tous les fichiers de tous les modèles : `active_storage_blobs` (un fichier : nom, type, taille, clé de stockage), `active_storage_attachments` (le lien entre un enregistrement et un fichier, par `record_type` et `record_id` : une association polymorphe) et `active_storage_variant_records` (les variantes d'une image, par exemple une miniature).

Joindre un fichier

```
class Book < ApplicationRecord
  has_one_attached :cover      # has_many_attached pour plusieurs fichiers
end
```

Aucune colonne dans `books`. Pour envoyer le fichier, trois changements :

```
params.expect(book: [ :title, :copies, :cover ])    # le contrôleur accepte le champ
```

```
<%= form.file_field :cover, accept: "image/png,image/jpeg" %>

<% if @book.cover.attached? %>
  <%= image_tag @book.cover %>
  <%= link_to "Télécharger", rails_blob_path(@book.cover, disposition: "attachment") %>
<% end %>
```

`form_with` passe seul le formulaire en `multipart/form-data` dès qu'il contient un `file_field`. `rails_blob_path` donne une URL de la forme `/rails/active_storage/blobs/redirect/eyJfcmlFp...-796b4f01.../germinal.png?` `disposition=attachment`, qui redirige vers le fichier ; `disposition: "attachment"` le fait télécharger plutôt qu'afficher.

En console ou dans les seeds :

```
file = File.open(Rails.root.join("db/seeds/germinal.png"))
book.cover.attach(io: file, filename: "germinal.png")
book.cover.attached?      # => true
book.cover.filename.to_s  # => "germinal.png"
book.cover.content_type   # => "image/png"
book.cover.byte_size      # => la taille, en octets
```

Où est le fichier ?

`config/storage.yml` déclare les services de stockage, et chaque environnement choisit le sien (`config.active_storage.service = :local` dans `config/environments/development.rb`). En développement, le service `local` écrit les fichiers sur le disque, dans `storage/`, sous un nom aléatoire (`storage/wv/sk/wvsk37g27pq98pwca9a8f5mby69t`). La base ne garde que les informations sur le fichier. En production, un autre service (Amazon S3, Google Cloud Storage, un disque partagé...) se déclare dans le même fichier, sans toucher au code de l'application.

Valider le type

`accept:` ne filtre que la fenêtre de sélection du navigateur, il ne vérifie rien côté serveur. Une validation, comme au step 07 :

```
class Book < ApplicationRecord
  validate :cover_is_an_image

  private
  def cover_is_an_image
    if cover.attached? && !%w[ image/png image/jpeg ].include?(cover.content_type)
      errors.add(:cover, "doit être une image PNG ou JPEG")
    end
  end
end
```

Un PDF envoyé à la place d'une image est refusé : « Cover doit être une image PNG ou JPEG ».

♦ Production et sécurité

Environnements. `RAILS_ENV` (`development`, `test` ou `production`) choisit le fichier `config/environments/<environnement>.rb` et la section de `config/database.yml`. En production, le code n'est pas rechargé à chaque modification, et les journaux vont sur la sortie standard.

Secrets. `bin/rails credentials:edit` ouvre `config/credentials.yml.enc`, chiffré et commité. La clé qui le déchiffre, `config/master.key`, n'est jamais commitée (elle est dans `.gitignore`) ; en production, elle passe par la variable `RAILS_MASTER_KEY`. Le code lit un secret avec `Rails.application.credentials.dig(:payment, :api_key)`.

Journaux. Les paramètres sensibles apparaissent en `[FILTERED]` dans les journaux (`"email" => "[FILTERED]"`) : la liste est dans `config/initializers/filter_parameter_logging.rb`.

Ce que Rails protège par défaut. Les requêtes forgées depuis un autre site (CSRF) : chaque formulaire porte un `authenticity_token`, vérifié à la réception (step 07). L'injection de HTML ou de JavaScript (XSS) : `<%= %>` échappe tout ce

qu'il affiche ; `raw` et `html_safe` désactivent l'échappement, à réserver à du HTML que vous avez écrit. L'affectation de champs non prévus : `params.expect` ne garde que les champs listés.

Ce qu'il ne protège pas : l'injection SQL. Une donnée reçue ne s'interpole jamais dans une chaîne SQL :

```
Book.where("title LIKE '%#{params[:q]}%")      # injectable
Book.where("title LIKE ?", "%#{params[:q]}%")  # Rails échappe la valeur
Book.where(title: params[:q])                  # la forme hash est sûre
```

Les outils. Le `Gemfile` d'une application Rails 8.1 contient déjà `brakeman` et `bundler-audit`. `bin/brakeman` analyse le code à la recherche de failles connues (injection SQL, XSS...); `bin/bundler-audit` compare les versions du `Gemfile.lock` à une base de failles publiées.

Déploiement. Une application Rails 8 est générée avec un `Dockerfile` et une configuration `Kamal` (`config/deploy.yml`) : `kamal deploy` construit l'image et la déploie sur vos serveurs. C'est un exemple parmi d'autres ; chaque hébergement a sa procédure.

♦ À reconnaître en lisant du code

Des notions que vous rencontrerez dans d'autres applications Rails :

Vous voyez	Ce que c'est
un callback que le modèle ne déclare pas	une macro ou une gem peut en poser : <code>has_one_attached</code> ajoute un <code>after_save</code> et un <code>after_commit</code> . <code>Book._save_callbacks</code> et <code>Book._commit_callbacks</code> les listent
<code>data-controller</code> , <code>data-action</code> dans une vue, des fichiers dans <code>app/javascript/controllers/</code>	Stimulus : le JavaScript que l'on écrit soi-même, accroché au HTML par des attributs
<code>app/mailers/</code> , <code>deliver_now</code> , <code>deliver_later</code>	Action Mailer : une classe par famille de mails, une méthode et des vues par mail, comme un contrôleur
<code>app/jobs/</code> , <code>perform_later</code>	Active Job : un traitement exécuté plus tard, hors de la requête ; en production, Rails 8 les confie à Solid Queue
<code>belongs_to :book</code> , <code>counter_cache: true</code>	Rails tient à jour le nombre d'emprunts dans une colonne <code>books.loans_count</code>
<code>Book.transaction do ... end</code> , <code>book.with_lock do ... end</code>	une transaction : toutes les écritures du bloc, ou aucune. <code>with_lock</code> verrouille en plus la ligne pendant le bloc

♦ Pour aller plus loin : la documentation officielle

Step 10 - Les tests

- [Testing Rails Applications](https://guides.rubyonrails.org/testing.html)
guides.rubyonrails.org/testing.html
- [Fixtures](https://guides.rubyonrails.org/testing.html#fixtures)
guides.rubyonrails.org/testing.html#fixtures
- [Testing Models](https://guides.rubyonrails.org/testing.html#testing-models)
guides.rubyonrails.org/testing.html#testing-models
- [Minitest assertions](https://docs.seattlerb.org/minitest/Minitest/Assertions.html)
docs.seattlerb.org/minitest/Minitest/Assertions.html
- [Integration Testing](https://guides.rubyonrails.org/testing.html#integration-testing)
guides.rubyonrails.org/testing.html#integration-testing

Step 11 - Des notes sur les ateliers et les sessions

- [Polymorphic Associations](https://guides.rubyonrails.org/association_basics.html#polymorphic-associations)
guides.rubyonrails.org/association_basics.html#polymorphic-associations
- [ActiveSupport::Concern](https://api.rubyonrails.org/classes/ActiveSupport/Concern.html)
api.rubyonrails.org/classes/ActiveSupport/Concern.html

Step 12 - Turbo Frames : modifier sur place

- [Turbo Drive - Form Submissions](https://turbo.hotwired.dev/handbook/drive#form-submissions)
turbo.hotwired.dev/handbook/drive#form-submissions
- [Turbo Frames](https://turbo.hotwired.dev/handbook/frames)
turbo.hotwired.dev/handbook/frames
- [Targeting Navigation Into or Out of a Frame](https://turbo.hotwired.dev/handbook/frames#targeting-navigation-into-or-out-of-a-frame)
turbo.hotwired.dev/handbook/frames#targeting-navigation-into-or-out-of-a-frame

Step 13 - Turbo Streams : s'inscrire sans recharger la page

- [Turbo Streams](https://turbo.hotwired.dev/handbook/streams)
turbo.hotwired.dev/handbook/streams
- [Streaming From HTTP Responses](https://turbo.hotwired.dev/handbook/streams#streaming-from-http-responses)
turbo.hotwired.dev/handbook/streams#streaming-from-http-responses
- [Broadcasting Page Refreshes](https://turbo.hotwired.dev/handbook/page_refreshes#broadcasting-page-refreshes)
turbo.hotwired.dev/handbook/page_refreshes#broadcasting-page-refreshes

Step 14 - Un support PDF par atelier

- [Active Storage Overview](https://guides.rubyonrails.org/active_storage_overview.html)
guides.rubyonrails.org/active_storage_overview.html
- [has_one_attached](https://guides.rubyonrails.org/active_storage_overview.html#attaching-files-to-records-has-one-attached)
guides.rubyonrails.org/active_storage_overview.html#attaching-files-to-records-has-one-attached